

Камењар

Баки наследил убаво парче земја во ридовите над Маркукуле. Инспириран од познатите куќи со камени кровови во градот, тој планира да изгради своја традиционална куќа.

Земјата е претставена како квадратна мрежа со димензии $N \times N$, каде редовите и колоните се нумерирани од 1 до N . Камениот терен има една необична особеност: во секој ред и во секоја колона од мрежата има точно една голема, неподвижна карпа.

За да ги постави темелите на својата куќа, Баки мора да избере **празна квадратна подмрежа** — регион со димензии $S \times S$ кој не содржи никакви карпи.

Нека M е максималниот цел број таков што, за *сиге* распореди на карпите што ги задоволуваат горенаведените услови, гарантирано постои празна квадратна подмрежа со димензии $M \times M$. Баки сака да најде *било која* празна квадратна подмрежа со димензии барем $M \times M$.

За да го премери земјиштето, Баки го бара својот пријател Пеки за помош. Пеки донел специјален уред за мерење наречен **Камењар** („бројач на камења“).

Во една операција, Пеки може да го конфигурира Камењарот да скенира правоаголна подмрежа зададена со нејзиниот горен-лев агол (x_1, y_1) и нејзиниот долен-десен агол (x_2, y_2) . Уредот го враќа точниот број на карпи во тој регион. Камењарот се напојува од мала батерија и секое скенирање троши многу енергија, па Пеки мора да го користи што е можно помалку пати.

Напишете програма која го користи Камењарот за да лоцира празна квадратна подмрежа со димензии $S \times S$ каде $S \geq M$, и ја пријавува нејзината локација.

Имплементациски детали

Треба да ја имплементирате следната процедура:

```
std::vector < int > solve(int N);
```

- N : големината на мрежата.
- Процедурата мора да врати точно четири цели броја $\{x_1, y_1, x_2, y_2\}$: горниот-лев и долниот-десен агол на празната квадратна подмрежа што ја нашла. Мора да важи

условот $x_2 - x_1 = y_2 - y_1 = S - 1$ со $S \geq M$.

- Оваа процедура се повикува точно еднаш за секој тест случај.

Процедурата може да прави повици до следната процедура:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : редот и колоната на горниот-лев агол на скенираната подмрежа.
- x_2, y_2 : редот и колоната на долниот-десен агол на скенираната подмрежа.
- Мора да важи $1 \leq x_1 \leq x_2 \leq N$ и $1 \leq y_1 \leq y_2 \leq N$. Инаку, вашата програма веднаш се прекинува и добива 0 поени за тој тест случај.
- Процедурата го враќа бројот на карпи во скенираната подмрежа.

Оценувачот не е адаптивен: позициите на карпите се фиксирани пред да се повика `solve` и не зависат од вашите повици до `ask`.

Ограничувања

- $2 \leq N \leq 500$
- Можете да направите најмногу $2.5 \cdot 10^5$ повици до `ask`.

Подзадачи

Подзадача	Поени	Дополнителни ограничувања
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Без дополнителни ограничувања.

Оценување

Ако `solve` врати невалиден регион, направи невалиден повик на `ask`, или ја повика `ask` повеќе од $2.5 \cdot 10^5$ пати, добивате 0 поени за тој тест случај.

Во спротивно, нека Q биде бројот на повици на `ask`. За тој тест случај ги добивате поените на подзадачата помножени со $\min(1, f(Q))$, каде што

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & \text{if } Q \leq 15 \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & \text{if } Q > 15 \end{cases}$$

Вашите поени за една подзадача се минималните поени по сите тест случаи во таа подзадача. Конечниот резултат е збир од поените на подзадачите.

За да ви помогне да ги процените вашите поени, табелата подолу го покажува приближниот множител за некои конкретни вредности на Q :

Број на прашања (Q)	Множител ($\approx$)
$Q \leq 11$	1.00
12	0.95
15	0.80
50	0.70
100	0.64
1000	0.45
10000	0.26
100000	0.07
$Q \geq 233573$	0.00

Пример-оценувач

Пример-оценувачот го чита влезот во следниот формат:

```
N
c_1 c_2 ... c_N
```

Тука, c_i е колоната на картата во редот i ($1 \leq c_i \leq N$).

Пример-оценувачот печати една линија што го содржи множителот $\min(1, f(Q))$ заработен со вашите прашања, или 0 ако вратената подмрежа или кој било повик на `ask` е невалиден.

Примери

Пример 1

Влез за пример-оценувачот:

```
4
1 2 3 4
```

Карпите се на $(1, 1)$, $(2, 2)$, $(3, 3)$ и $(4, 4)$. Вашата програма не го знае ова.

Оценувачот ја повикува `solve(4)`. Една можна низа од повици е:

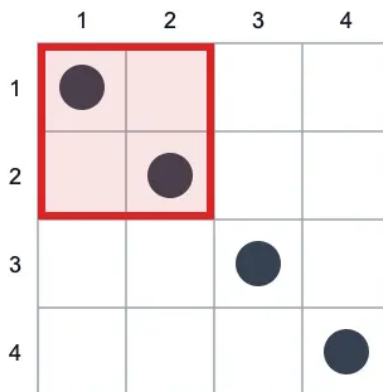
- `ask(1, 1, 2, 2)` враќа 2.
- `ask(2, 4, 4, 4)` враќа 1.
- `ask(3, 1, 3, 2)` враќа 0.

Потоа `solve` враќа $\{1, 3, 2, 4\}$.

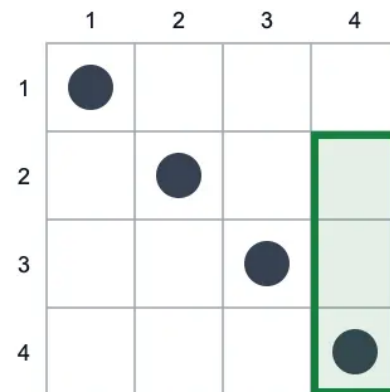
Излез на пример-оценувачот:

```
1
```

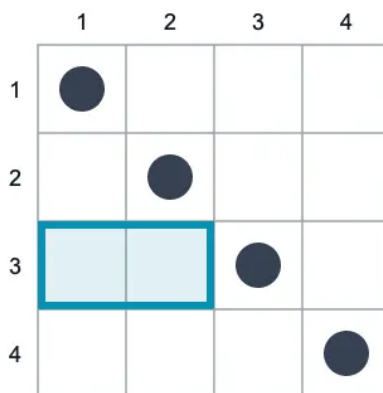
Првиот повик го скенира горно-левиот блок 2×2 и ги наоѓа карпите на $(1, 1)$ и $(2, 2)$. Вториот ги скенира редовите од 2 до 4 од колоната 4 и ја наоѓа карпата на $(4, 4)$. Третиот ги скенира првите две полиња од редот 3 и не наоѓа карпи. Вратениот регион ги покрива редовите од 1 до 2 и колоните од 3 до 4, што е празна подмрежа 2×2 .



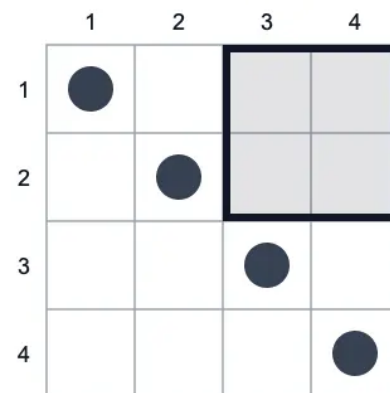
ask(1, 1, 2, 2) returns 2



ask(2, 4, 4, 4) returns 1



ask(3, 1, 3, 2) returns 0



return {1, 3, 2, 4}

За $N = 4$, програмата искористи $Q = 3$ повици, па $f(Q) = 1.4$ и множителот е 1. Прифатливи се и други одговори, на пример $\{3, 1, 4, 2\}$, $\{1, 2, 1, 2\}$, $\{3, 2, 3, 2\}$, $\{3, 4, 3, 4\}$ или кое било празно поле бидејќи $M = 1$ за $N = 4$.